

JavaScript Functions Cheat Sheet

Practical reference for array, string, object, and async methods — when to use each, and performance notes for real backend work (Node/Express/Mongoose).

1. Array Methods — Reading/Transforming (non-mutating)

These return a **new** array/value and don't change the original — generally preferred, since mutating shared data causes hard-to-trace bugs.

Method	Example	When to use	Performance note
<code>.map()</code>	<code>arr.map(x => x * 2)</code>	Transform every item into something new, same length out as in.	O(n). Don't use if you're not using the returned array — use <code>.forEach()</code> instead.
<code>.filter()</code>	<code>arr.filter(x => x > 5)</code>	Keep only items matching a condition. Returns a possibly-shorter array.	O(n). Combine with <code>.map()</code> sparingly — chaining both means iterating twice; a single <code>.reduce()</code> can do both in one pass if performance is critical.
<code>.reduce()</code>	<code>arr.reduce((sum, x) => sum + x, 0)</code>	Collapse an array into a single value (sum, object, grouped data, etc). Most flexible/powerful array method.	O(n), single pass. Harder to read for beginners — use <code>.map()</code> / <code>.filter()</code> first unless reduce genuinely simplifies the logic.
<code>.find()</code>	<code>arr.find(x => x.id === targetId)</code>	Get the first item matching a condition, or <code>undefined</code> .	O(n) worst case, but stops early once found — faster than <code>.filter()[0]</code> which always scans the whole array.
<code>.findIndex()</code>	<code>arr.findIndex(x => x.id === id)</code>	Same as <code>.find()</code> but returns the index , not the item. Returns <code>-1</code> if not found.	Same early-exit behavior as <code>.find()</code> .
<code>.some()</code>	<code>arr.some(x => x.age > 18)</code>	Check if at least one item matches — returns <code>true</code> / <code>false</code> .	Stops at first match — faster than <code>.filter().length > 0</code> .

Method	Example	When to use	Performance note
<code>.every()</code>	<code>arr.every(x => x.age > 18)</code>	Check if all items match — returns <code>true</code> / <code>false</code> .	Stops at first failure — efficient for validation checks.
<code>.includes()</code>	<code>arr.includes(5)</code>	Check if an array contains an exact value (primitives only).	O(n). For objects, <code>.includes()</code> checks reference equality — use <code>.some()</code> instead for object matching.
<code>.slice()</code>	<code>arr.slice(0, 3)</code>	Extract a portion of an array without modifying the original.	Non-mutating — always prefer this over <code>.splice()</code> unless you specifically need mutation.
<code>.flat()</code>	<code>arr.flat()</code>	Flattens nested arrays by one level (<code>.flat(Infinity)</code> for full flatten).	Rarely a bottleneck; used for cleaning up nested data structures.
<code>.sort()</code>	<code>arr.sort((a, b) => a - b)</code>	Sorts an array. Mutates the original array!	Always provide a comparator function for numbers — <code>arr.sort()</code> alone treats numbers as strings (<code>10</code> sorts before <code>2</code>).

2. Array Methods — Looping (mutating / side-effect focused)

Method	Example	When to use	Performance note
<code>.forEach()</code>	<code>arr.forEach(x => console.log(x))</code>	Run side effects (logging, pushing to an outside variable) — no return value used.	Does NOT wait for async callbacks — never use with <code>await</code> inside. Use <code>for...of</code> instead for async work.
<code>for...of</code>	<pre>for (const x of arr) { await doSomething(x); }</pre>	The correct loop for sequential async operations — each iteration properly waits.	Slower than <code>Promise.all()</code> for independent async tasks, but necessary when order matters or later steps depend on earlier ones.
<code>.push()</code> / <code>.pop()</code>	<code>arr.push(item)</code>	Add/remove from the end of an array.	O(1) — fast, no reindexing needed.

Method	Example	When to use	Performance note
<code>.shift()</code> / <code>.unshift()</code> <code>arr.unshift(item)</code>		Add/remove from the start of an array.	O(n) — every other element has to shift index. Avoid in large arrays/loops if avoidable.
<code>.splice()</code>	<code>arr.splice(2, 1)</code>	Remove/insert items at a specific index. Mutates the original array.	Use <code>.slice()</code> + spread instead if you want to avoid mutating shared/state data (e.g. in React).

3. String Methods

Method	Example	When to use	Performance note
<code>.split()</code>	<code>"a,b,c".split(",")</code>	Turn a string into an array, breaking on a delimiter.	Fine for typical use; avoid on huge strings in hot loops.
<code>.join()</code>	<code>["a","b"].join("-")</code>	Turn an array back into a string with a separator.	Much faster than string concatenation (<code>+=</code>) in a loop for building large strings.
<code>.slice()</code>	<code>"hello".slice(1, 3)</code>	Extract part of a string by index. Accepts negative indexes (<code>slice(-3)</code> = last 3 chars).	Preferred over <code>.substring()</code> — more predictable with negative numbers.
<code>.trim()</code>	<code>" hi ".trim()</code>	Remove leading/trailing whitespace — essential before saving user input.	Cheap operation, use freely on all user-submitted text fields.
<code>.includes()</code>	<code>"hello world".includes("world")</code>	Check if a substring exists — clearer than <code>.indexOf() !== -1</code> .	O(n), fine for normal string lengths.
<code>.replace()</code>	<code>"foo bar".replace("foo", "baz")</code>	Replace first match. Use <code>.replaceAll()</code> for every match, or a regex with <code>/g</code> flag.	Regex-based replace is more powerful but slightly slower — use plain string replace when you don't need patterns.

Method	Example	When to use	Performance note
<code>.charAt()</code>	<code>"hello".charAt(0)</code>	Get character at index. Never throws — returns <code>" "</code> if out of range.	Safer than <code>str[0]</code> when the string might be empty (<code>str[0]</code> returns <code>undefined</code> , <code>.charAt(0)</code> returns <code>" "</code>).
<code>.toUpperCase()</code> <code>.toLowerCase()</code>	<code>"abc".toUpperCase()</code>	Case conversion — used constantly for normalizing emails/ usernames before saving.	Cheap; always normalize case before uniqueness checks (<code>findOne({ email })</code>) to avoid case-sensitive duplicate bugs.
<code>.padStart()</code> <code>.padEnd()</code>	<code>"5".padStart(2, "0")</code> <code>// "05"</code>	Pad a string to a fixed length — common for formatting IDs, times, invoice numbers.	Cheap, no real performance concern.
<code>.repeat()</code>	<code>"ab".repeat(3) → "ababab"</code>	Repeat a string N times.	Rarely a bottleneck.

4. Object Methods

Method	Example	When to use	Performance note
<code>Object.keys()</code>	<code>Object.keys(user)</code>	Get an array of a plain object's keys.	O(n) where n = number of keys — fine for typical objects.
<code>Object.values()</code>	<code>Object.values(user)</code>	Get an array of a plain object's values.	Same cost as <code>.keys()</code> .
<code>Object.entries()</code>	<code>Object.entries(user)</code>	Get <code>[key, value]</code> pairs — great for looping with <code>for...of</code> .	Common pattern: <code>for (const [key, val] of Object.entries(obj))</code> .
<code>Object.assign()</code>	<code>Object.assign({}, defaults, overrides)</code>	Merge objects (shallow). Later sources overwrite earlier ones.	Shallow only — nested objects get replaced whole, not deep-merged. Spread (<code>{...a, ...b}</code>) is equivalent and more common now.

Method	Example	When to use	Performance note
<code>{...obj}</code> (spread)	<code>{ ...user, isOnline: true }</code>	Create a shallow copy of an object with some fields overridden.	Preferred modern syntax over <code>Object.assign()</code> — same performance, cleaner to read.
<code>Object.freeze()</code>	<code>Object.freeze(config)</code>	Prevent an object from being modified (constants, config objects).	Shallow freeze only — nested objects inside can still be mutated unless frozen individually.

5. Async / Promise Patterns

Pattern	Example	When to use	Performance note
<code>for...of + await</code>	<code>for (const id of ids) { await Model.findById(id); }</code>	Sequential — each operation waits for the previous one. Use when order matters or each step depends on the last.	Slowest for independent operations — N operations take $N \times (\text{time per operation})$, since nothing runs in parallel.
<code>Promise.all()</code>	<code>await Promise.all(ids.map(id => Model.findById(id)))</code>	Run independent async operations in parallel . Use when order doesn't matter and operations don't depend on each other.	Much faster for independent DB calls — N operations take roughly the time of the <i>slowest single one</i> , not the sum. Fails entirely if even one promise rejects.
<code>Promise.allSettled()</code>	<code>await Promise.allSettled(promises)</code>	Like <code>Promise.all()</code> , but doesn't fail-fast — gives you the result/error of every promise, even if some failed.	Use when partial failure is acceptable (e.g. sending notifications to 10 users — one failing shouldn't stop the other 9).

Pattern	Example	When to use	Performance note
<code>.forEach()</code> + <code>async</code>	<code>arr.forEach(async x => await f(x))</code>	Never use this.	Silently broken — <code>.forEach()</code> doesn't wait for the async callbacks, so code after it runs before the async work finishes.

6. Quick Decision Guide

I want to...	Use
Transform every item into something new	<code>.map()</code>
Keep only items matching a condition	<code>.filter()</code>
Get one value out of the whole array (sum, grouped object, etc.)	<code>.reduce()</code>
Find the first matching item	<code>.find()</code> (not <code>.filter()[0]</code> — slower)
Check “does at least one match”	<code>.some()</code>
Check “do all match”	<code>.every()</code>
Loop with side effects only, no async	<code>.forEach()</code>
Loop with <code>await</code> inside	<code>for...of</code> (never <code>.forEach()</code>)
Run many independent async calls fast	<code>Promise.all()</code>
Run many async calls where some are allowed to fail	<code>Promise.allSettled()</code>
Copy an object with a few fields changed	<code>{ ...obj, field: newValue }</code>
Build a string from array pieces	<code>.join()</code> (not <code>+=</code> in a loop)